

1. What is Utility-first? ✨

You style elements by combining small, single-purpose classes directly in HTML/JSX.

Each class does one thing well.

```
<button class="px-4 py-2
bg-blue-600 text-white
rounded-lg">Click</button>
```

2. Why it matters ❤️

- ⚡ Fast styling
- 🎯 Consistency across UI
- 📄 Less custom CSS
- 🔧 Easier maintenance
- 👥 Great for collaboration

3. How to read a class 🔍

Most classes follow the pattern:

utility-group - value

Example:

text-blue-600

utility group value (option)

Here: text (color group) with blue-600 value.

4. Reading pattern 📖

Classes can include variants before the utility and value.

breakpoint : state : utility - value

Example:

md:hover:bg-blue-600

breakpoint (size) state (condition) utility value

5. Spacing utilities 🌊

- p-4 padding all sides
- px-6 padding left & right
- py-2 padding top & bottom
- m-4 margin all sides
- mt-2 margin top
- gap-4 space between items

Scale: 0, 0.5, 1, 1.5, 2, 3, 4, 5, 6, 8, 10, 12, 16, 20, ...

6. Sizing utilities 📏

- w-64 width (16rem)
- h-10 height (2.5rem)
- min-h-screen minimum height
- max-w-md max width

Use fixed, min-*, max-*, full (w-full, h-full), or auto.

7. Container & layout width 📐

- container set a responsive max-width at each breakpoint
- mx-auto center a block element horizontally
- max-w-* control the max width (e.g., max-w-7xl)

8. Typography 📖

- text-sm small text
- text-xl extra large
- font-bold bold weight
- leading-6 line height
- text-center center align

More: italic, uppercase, tracking-wide, etc.

9. Colors 🎨

- text-blue-600 text color
- bg-slate-100 background color
- border-gray-300 border color

Use color shades: -50 to -950.

10. Borders & radius 📐

- border 1px solid border
- border-2 2px border
- rounded small radius
- rounded-xl large radius

More: rounded-sm, md, 2xl, full

11. Shadows ☁️

- shadow small shadow
- shadow-md medium shadow
- shadow-lg large shadow

More: shadow-sm, xl, 2xl, inner (inner shadow).

12. Display & spacing flow 📑

- block block element
- inline-block inline-block
- hidden display: none
- space-y-4 vertical space between children

13. Flex basics 🧑‍🤝🧑

- flex create flex container
- items-center align items vertically
- justify-between space items horizontally
- flex-col stack items vertically
- gap-3 space between items

14. Grid basics 📐

- grid create grid container
- grid-cols-2 2 equal columns
- md:grid-cols-4 4 cols on md+
- gap-4 gap between grid items

15. Responsive design 📺

- sm: small ≥ 640px
- md: medium ≥ 768px
- lg: large ≥ 1024px
- xl: extra large ≥ 1280px
- 2xl: 2 extra large ≥ 1536px

Utilities can change by screen size.

16. State variants 🖱️

- hover: on hover
- focus: when focused
- active: when active
- disabled: when disabled

Examples:

- hover:bg-blue-700
- focus:ring-2

17. Example button 📄

```
<button class="bg-blue-600
text-white px-4 py-2
rounded-lg hover:bg-blue-700
focus:ring-2 focus:ring-blue-400
transition">
  Save
</button>
```

Save

18. Example card 📄

```
<div class="max-w-sm p-4
rounded-xl shadow-md bg-white
border border-gray-200">
  <h3 class="text-lg font-bold
mb-2">Card title</h3>
  <p class="text-gray-600">
    Short description here. </p>
</div>
```

Card title

Short description here.

19. How to think in Tailwind 🧠

- Layout (display, flex/grid, width)
 - Spacing (padding, margin, gap)
 - Typography (size, weight, align)
 - Color (text, background, border)
 - Effects (radius, shadow, ring)
- Read left to right.

20. Common rule 💡

If you can describe a visual change, Tailwind probably has a utility for it.

Think in utilities, not custom CSS.

21. Quick do's ✅

- ✓ Keep classes grouped logically.
- ✓ Reuse repeated patterns with components.

22. Key takeaway

Tailwind = many small utilities combined to build UI quickly, consistently, and confidently.

درس ۲: Utility-first و روش خواندن کلاس‌ها

خلاصه درس دوم — منطق Tailwind و ترکیب Utility Class ها

۱) Utility چیست؟

هر Utility Class یک وظیفه مشخص دارد.

```
<h1 className="text-4xl font-bold text-blue-600">
```

- text-4xl → اندازه متن
- font-bold → وزن فونت
- text-blue-600 → رنگ متن

۲) Utility-first یعنی چه؟

به جای ساختن کلاس‌های بزرگ، چند Utility کوچک را کنار هم قرار می‌دهیم.

سنتی CSS

```
.save-button { ... }
```

Tailwind

```
<button className="bg-blue-600 text-white  
px-4 py-2 rounded-lg">
```

۳) روش خواندن کلاس‌ها

```
bg-white p-6 rounded-xl shadow-lg
```



bg-white

→ پس‌زمینه



p-6

→ فاصله داخلی



rounded-xl

→ گوشه گرد



shadow-lg

→ سایه

هر بخش از className یک دستور کوچک است.

۴) ساختار نام‌گذاری

بیشتر کلاس‌ها الگوی property-value دارند.

property-value

bg-blue-600

text-xl

rounded-lg

shadow-md

۵) خانواده‌های مهم Utility



bg- → background



text- → متن / رنگ یا اندازه



font- → فونت



p- / m- → فاصله‌ها



w- / h- → عرض و ارتفاع



rounded- → گوشه‌ها



shadow- → سایه



flex / grid → چیدمان

در Tailwind، رابط کاربری از کنار هم قرار گرفتن Utility ها ساخته می‌شود.



ادامه Tailwind و Utility-first

variantها، className و مدل ذهنی ساخت رابط کاربری

Variant چیست؟ (۶ ⚡)

Variant یعنی Utility فقط در یک حالت یا شرط خاص اعمال شود.

hover:

focus:

sm:

md:

dark:

```
<button className="bg-blue-500 hover:bg-blue-700">
```

- تغییر رنگ در حالت `Hover:bg-blue-700` →
- اعمال کلاس از اندازه small به بالا `sm:text-lg`

class و className (۷ </>)

React JSX

```
<div className="bg-blue-600"></div>
```

HTML

```
<div class="bg-blue-600"></div>
```

در React به جای class از className استفاده می‌کنیم.

ترکیب Utilityها (۸ ≡)

```
<div className="bg-blue-600 text-white p-6 rounded-xl shadow-lg">
```

shadow-lg

→ سایه

rounded-xl

→ گوشه گرد

p-6

→ فاصله داخلی

text-white

→ رنگ متن

bg-blue-600

→ پس‌زمینه

رابط کاربری با کنار هم قرار گرفتن Utilityها ساخته می‌شود.

مدل ذهنی Tailwind (۹ 🏗️)

Element

→ className

→ Utilityها

→ Tailwind

→ CSS

→ UI

Tailwind کلاس‌ها را از فایل‌ها می‌خواند و CSS لازم را تولید می‌کند.

نکات مهم درس ۲ (۱۰ 📋)

✓ هر Utility یک وظیفه مشخص دارد.

✓ به جای کلاس‌های بزرگ، Utilityهای کوچک را ترکیب می‌کنیم.

✓ بیشتر کلاس‌ها الگوی property-value دارند.

✓ Variantها برای حالت‌ها و شرط‌ها هستند.

✓ در JSX از className استفاده می‌شود.

✓ در Tailwind شناخت الگوها مهم‌تر از حفظ کردن همه کلاس‌هاست.

مرحله بعد: Sizing , Spacing و Container

